

CSE-4112: Artificial Intelligence Sessional

Brief Introduction to Prolog (Part 2)



SWI Prolog



OneCompiler



GNU Prolog

Instructor: Shafkat Kibria

Leading University, Sylhet

(slides adapted from "CSC384: Intro to Artificial Intelligence" - Hojjat Ghaderi and Fahiem Bacchus, University of Toronto)

More advanced material

- Debugging Prolog programs.
- Passing predicates as arguments and constructing predicates dynamically (on-the-fly).
- Efficient lists processing using accumulators.
- Negation as failure (NAF).
- Cut (controlling how Prolog does the search)
- if-then-else

More advanced material: Debugging Prolog Programs

- Text-based debugger:
 - You can put or remove a breakpoint on a predicate:
 - `?-spy(female).`
 - `?-nospy(female).`
 - To start debugging use “trace” before the query:
 - `?-trace, male(X).`
 - While tracing you can do the following:
 - `creap`: step inside the current goal (press c/enter/or space)
 - `leap`: run up to the next spy point (press l)
 - `skip`: step over the current goal without debugging (press s)
 - `abort`: abort debugging (press a)
 - And much more... press h for help

Text Debugger Example

- ?-spy(female/1).

yes

- ?-mother(X,Y). %starts debugging!

Call: (9) female(albert) ?

- ?-nospy(female/1).

% Spy point removed from female/1

- trace, father(X,Y). %let's debug!

Call: (9) father(_G305, _G306) ?

```
male(albert).  
male(edward).
```

```
female(alice).  
female(victoria).
```

```
parent(albert,edward).  
parent(victoria,edward).
```

```
father(X,Y):- parent(X,Y), male(X).
```

```
mother(X,Y):- parent(X,Y), female(X).
```

Simple Exercise: debug this program both in the text

Passing Predicates as Argument

- We can pass a predicate as an argument to a rule:
test(X) :-X.
?-test(male(john)). %succeeds if male(john) holds.
?-test(parent(carrot,4)). %fails .
- What if we want to pass arguments of the predicate separately?
test(X,Y) :-X(Y). % this is syntax error!
?-test(male, john).
- Unfortunately the above does not work, we cannot write X(Y) !!
- =.. is used to build predicates on the fly:
test(X,Y) :-G =.. [X,Y], G. %builds predicate X(Y) dynamically and calls it
?-test(male, john).
- In general, G =.. [P,X1,X2,...,Xn] creates P(X1,X2,...,Xn). E.g:
?-G =.. [parent, john, X].
G = parent(john, X)

Adding/Deleting Rules/Facts Dynamically

- A program can add or delete facts/rules dynamically:
 - `assert(term)` %adds the given rule or fact
 - `assert(male(john)).`
 - `assert(dog(tom)).`
 - `assert((animal(X) :-dog(X))).`
 - `retract(term)` %deletes the first fact/rule that unifies with the given term
 - `retract(dog(tom)).`
 - `retract((animal(X) :-dog(X))).`
 - `retract(animal(_)).`
 - `died(X), retract(parent(john,X)).`
 - `retractall(term)` %deletes ALL facts/rules that unify
 - `retractall(parent(_,_)).`
 - There is also `assertz(term)` that adds the fact/rule to the end rather than beginning

More Lists Processing in Prolog

- Much of Prolog's computation is organized around lists.
- Many built-in predicates: `member`, `append`, `length`, `reverse`.
- List of lists:
 - `[[1,2], [a, b, c(d), 4], []]`
 - Can define a matrix, e.g. `3x2 M=[[1,2], [-1,4], [5,5]]`
 - Elements can be structures: `M2= [[(1.5,a), (3.2,b)], [(0,c), (7.2,d)]]` is a 2x2 matrix.

Then if write `M2=[H | _]`, `H=[_,(Cost, Name)]`. It succeeds and we get `Cost=3.2` and `Name=b`.

- Exercise: write a predicate `getElm(M,R,C,E)` which holds if element `E` is at `M[R][C]`. Assume the matrix is `KxL` and `I,J >= 0` and in range. Note that `M[0][0]` is the element at 1st row 1st column

Lists: Extracting Desired Elements

- Extracting all elements satisfying a condition: e.g. `extract(male, [john, alice, 2, sam], Males)`
- Generally: `extract(+Cond, +List, ?Newlist)`

%Note: +, -, ? are not actual Prolog symbols, just used as convention!

`extract(_,[ ], [ ])`.

`extract(Condname, [X | Others], [ X | Rest]):-`

`Cond =..[Condname,X], %building cond predicateCondName(X)`

`Cond, %now, calling Cond predicate to see if it holds`

`extract(Condname, Others, Rest).`

`extract(Condname, [X | Others], Rest):-Cond =..[Condname,X],`

`\+Cond, extract(Condname, Others, Rest).`

\+ is negation as failure. We can also simplify the above using if-then-else. We will get back to these in a couple of slides.

Lists: append

- Appending two lists (challenge: no assumption on args):

`append(?X, ?Y, ?Z)`

Holds iff Z is the result of appending lists X and Y.

- Examples:

- `append([a,b,c],[1,2,3,4],[a,b,c,1,2,3,4])`

- Extracting the third element of L: `append([_,_,X],_,L)`

- Extracting the last element of L: `axppend(_,[X],L)`

- Finding two consecutive elements X&Y in L:

`append(_,[X,Y | _],L)`

Implementing append

- definition: `append(?X, ?Y, ?Z)`

`append([ ],L,L).`

`append([H | T],L,[H | L2]):-`

`append(T,L,L2).`

- What are all the answers to `append(_,[X,Y],[1,2,3,4,5])` ?
- What are all the answers to `append(X,[a],Y)` ?
- What is the answer to `append(X,Y,Z)`? How many answers?

Lists: reversing

- Reversing a list: $[1,2,[a,b],3] \rightarrow [3,[a,b],2,1]$

`reverse(?L,?RevL)`

`reverse([ ], [ ]).`

`reverse([H | T],RevL):-`

`reverse(T,RevT), append(RevT,[H],RevL).`

- This is not efficient! Why? $O(N^2)$

Efficiency issues: Fibonacci

- Fibonacci numbers:
$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$
- Consider the following implementation:
fib(0,0).
fib(1,1)
fib(N,F):- N>1,
N1 is N-1, fib(N1,F1), N2 is N-2, fib(N2, F2),
F is F1+F2.
- This is very inefficient (exponential time)! Why?
- Solution: use **accumulator**!